NinjaRS benchmark

Batched one-loop integrand reduction in Rust versus MadLoop (Ninja + OneLOop)

2026-10-17 · AMD EPYC 9754 (shared host), one pinned core

Reading guide. Every multiplier is *NinjaRS time ÷ reference time* for the same work on the same inputs: generation is compared with MadLoop output plus compilation; the numerator with MadLoop's loop-coefficient computation (including Ninja's tensor-numerator expansions); the pure reduction with original Ninja; the master integrals with OneLOop; the total with the complete MadLoop pipeline. Green cells mean NinjaRS is faster, red slower; white is parity. NinjaRS evaluates batches of 32 phase-space points; times are per point.

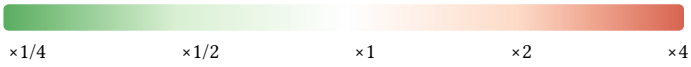
Performance gates (binary64, NinjaRS batched; ranges over the physical diagrams of Table 1, identical-input values from Table 2)

(a) Pure reduction	NinjaRS ≤ original Ninja on identical inputs	**×0.47–0.59**
(b) Master integrals	oneloopmaster native ≤ 1.3 × OneLOop on the same calls	**×0.29–0.71**
(c) End to end	NinjaRS (SymJIT O2 numerator + reduction + masters) ≤ MadLoop pipeline	**×0.18–0.52**

1 Summary of all test cases

Table 1: **Physical diagrams versus the MadLoop pipeline.** Generation time and per-phase-space-point evaluation time (binary64, one helicity configuration, a single loop diagram). Each cell shows the NinjaRS value in bold, the reference value below it and the multiplier NinjaRS ÷ reference. Generation: NinjaRS CLI graph preparation, coefficient planning and SymJIT O2 compilation versus MadLoop process output plus compilation (ML). Numerator: SymJIT O2 coefficient heads and tensor-numerator expansions versus MadLoop's loop-coefficient construction and Ninja's tensor-numerator expansions (ML). Reduction: pure reduction versus original Ninja inside MadLoop (N). Masters: oneloopmaster's native code versus OneLOop (O). Total: complete evaluation versus the complete MadLoop pipeline (ML). The rank column gives NinjaRS's rank and, in brackets, the rank MadLoop passes to Ninja.

Test case	n / r	Generation [s]	Numerator [µs]	Reduction [µs]	Masters [µs]	Total [µs]
gg → H top triangle	3 / 2 (3)	**0.143** ML 4.26 ×0.03	**0.942** ML 14.3 ×0.07	**1.28** N 4.08 ×0.31	**1.97** O 3.26 ×0.60	**4.25** ML 21.4 ×0.20
γγ → γγ top box	4 / 4 (4)	**5.16** ML 5.19 ×1.00	**19.8** ML 41.2 ×0.48	**5.24** N 10.5 ×0.50	**5.67** O 9.08 ×0.62	**31.1** ML 59.9 ×0.52
gg → HH top triangle	3 / 2 (3)	**0.143** ML 4.57 ×0.03	**0.913** ML 12.6 ×0.07	**1.30** N 3.88 ×0.33	**0.985** O 1.97 ×0.50	**3.21** ML 18.3 ×0.18
gg → HH top box	4 / 4 (4)	**0.698** ML 4.95 ×0.14	**11.2** ML 34.8 ×0.32	**5.38** N 11.5 ×0.47	**13.8** O 32.1 ×0.43	**30.6** ML 78.2 ×0.39
gg → HHH top triangle	3 / 2 (3)	**0.144** ML 5.66 ×0.03	**1.04** ML 12.6 ×0.08	**1.29** N 3.86 ×0.33	**0.872** O 1.25 ×0.70	**3.22** ML 17.5 ×0.18
gg → HHH top box	4 / 4 (4)	**1.49** ML 6.04 ×0.25	**11.1** ML 34.7 ×0.32	**5.39** N 11.7 ×0.46	**12.9** O 27.2 ×0.47	**29.7** ML 73.4 ×0.40
gg → HHH top pentagon	5 / 4 (5)	**2.80** ML 6.83 ×0.41	**15.3** ML 105 ×0.15	**9.04** N 26.0 ×0.35	**55.2** O 105 ×0.53	**80.8** ML 235 ×0.34



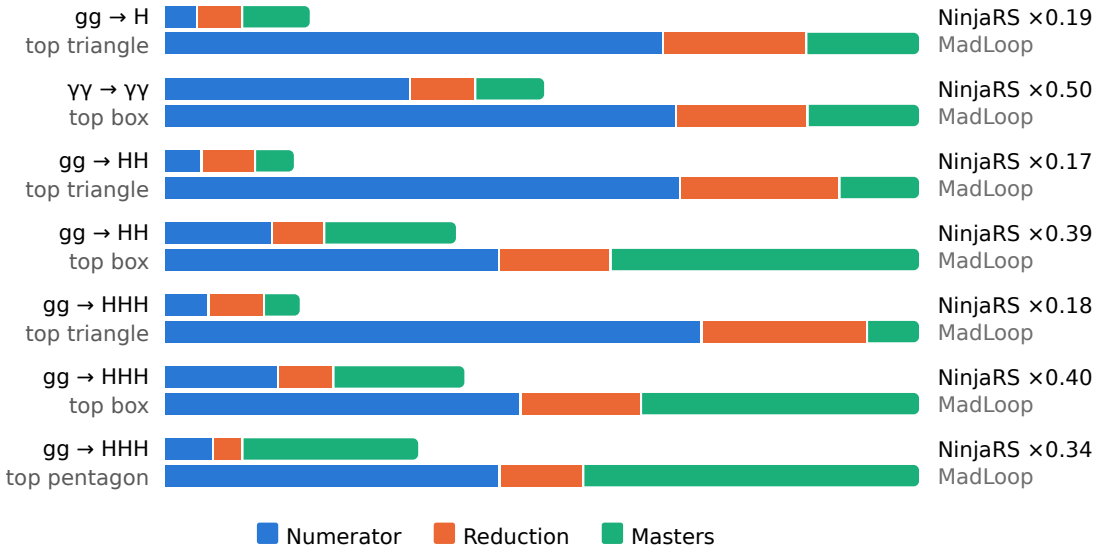


Figure 1: Per-point evaluation time of each diagram, NinjaRS (upper bar, batched) and the MadLoop pipeline (lower bar), stacked by component and normalized to the MadLoop total. The label gives NinjaRS's total as a multiple of MadLoop's; Table 1 lists the values.

Table 2: **Identical inputs versus original Ninja and OneLOop.** Both implementations receive the same tensor-numerator coefficients, propagators and invariants (binary64), and evaluate the same rank. Tensor numerator: Ninja-layout tensor expansions in NinjaRS's batched `TensorNumerator` versus Ninja's `TensorNumerator` (N); pure reduction with replayed numerator and master values (N); master integrals on the identical call stream, oneloopmaster native versus OneLOop (O); total with live numerator and masters. Original Ninja is built with `g++ -O3 -fcx-fortran-rules` (MadLoop's flag; 20 % faster than plain `-O3`). The last column is the maximal difference of the three Laurent coefficients relative to the largest, NinjaRS versus Ninja.

Case	n / r	Tensor num. [µs]	Reduction [µs]	Masters [µs]	Total [µs]	Δ vs Ninja
gg → H, top triangle	3 / 2	0.435 N 0.902 ×0.48	1.27 N 2.61 ×0.49	1.88 O 6.54 ×0.29	3.59 N+O 10.1 ×0.36	1.1e−14
γγ → γγ, top box	4 / 4	9.55 N 22.7 ×0.42	5.31 N 9.10 ×0.58	5.80 O 10.1 ×0.57	20.8 N+O 42.7 ×0.49	1.7e−12
gg → HH, top triangle	3 / 2	0.426 N 0.883 ×0.48	1.28 N 2.55 ×0.50	0.983 O 1.88 ×0.52	2.69 N+O 5.38 ×0.50	2.7e−15
gg → HH, top box	4 / 4	9.58 N 22.8 ×0.42	5.42 N 9.25 ×0.59	13.6 O 27.1 ×0.50	28.5 N+O 60.4 ×0.47	5.8e−13
gg → HHH, top triangle	3 / 2	0.428 N 0.901 ×0.47	1.29 N 2.54 ×0.51	0.897 O 1.26 ×0.71	2.61 N+O 4.78 ×0.55	2.0e−15
gg → HHH, top box	4 / 4	9.47 N 22.6 ×0.42	5.39 N 9.30 ×0.58	12.9 O 22.0 ×0.59	27.8 N+O 54.8 ×0.51	1.5e−14
gg → HHH, top pentagon	5 / 4	11.4 N 22.0 ×0.52	8.87 N 19.0 ×0.47	54.2 O 91.4 ×0.59	75.2 N+O 134 ×0.56	1.1e−14
Synthetic topologies (random kinematics, real and complex masses, dense random tensors)						
`syn-gghhh-gHgHH-n5-r4`	5 / 4	11.3 N 22.5 ×0.50	8.94 N 18.5 ×0.48	29.1 O 60.8 ×0.48	50.7 N+O 104 ×0.49	1.2e−12
`syn-gghhh-gHgHH-n5-r5`	5 / 5	36.2 N 81.0 ×0.45	13.3 N 23.4 ×0.57	28.4 O 59.1 ×0.48	78.3 N+O 166 ×0.47	1.4e−11
`syn-gghhh-ggHHH-n5-r4`	5 / 4	11.4 N 22.4 ×0.51	8.68 N 18.4 ×0.47	59.9 O 110 ×0.54	80.6 N+O 153 ×0.53	2.9e−12
`syn-gghhh-ggHHH-n5-r5`	5 / 5	36.1 N 81.7 ×0.44	13.0 N 22.5 ×0.58	60.3 O 110 ×0.55	110 N+O 218 ×0.50	9.0e−3
`syn-gghhh-nosmat-n5-r4`	5 / 4	11.5 N 22.5 ×0.51	8.78 N 18.6 ×0.47	60.7 O 110 ×0.55	81.3 N+O 154 ×0.53	3.7e−12
`syn-n2-r1`	2 / 1	0.029 N 0.062 ×0.46	0.260 N 0.422 ×0.62	0.253 O 0.212 ×1.19	0.542 N+O 0.665 ×0.82	4.0e−15
`syn-n2-r2`	2 / 2	0.203 N 0.435 ×0.47	0.282 N 0.462 ×0.61	0.264 O 0.195 ×1.35	0.743 N+O 1.08 ×0.69	1.9e−13
`syn-n2-r3`	2 / 3	1.04 N 2.71 ×0.38	0.511 N 0.978 ×0.52	0.261 O 0.307 ×0.85	1.80 N+O 4.04 ×0.45	3.5e−11
`syn-n3-r2`	3 / 2	0.445 N 0.933 ×0.48	1.30 N 2.51 ×0.52	2.54 O 4.28 ×0.59	4.28 N+O 7.72 ×0.55	7.3e−14

Case	n / r	Tensor num. [µs]	Reduction [µs]	Masters [µs]	Total [µs]	Δ vs Ninja
syn-n3-r3	3 / 3	1.82 N 4.67 ×0.39	1.71 N 2.96 ×0.58	2.54 O 3.85 ×0.66	6.08 N+O 11.6 ×0.52	4.8e−14
syn-n3-r4	3 / 4	7.77 N 20.3 ×0.38	4.06 N 7.21 ×0.56	2.55 O 4.98 ×0.51	14.4 N+O 32.8 ×0.44	6.2e−14
syn-n4-r3	4 / 3	2.55 N 5.36 ×0.48	3.41 N 7.08 ×0.48	12.5 O 16.7 ×0.75	18.5 N+O 29.5 ×0.63	1.3e−1
syn-n4-r4	4 / 4	9.29 N 22.5 ×0.41	5.07 N 8.68 ×0.58	14.1 O 15.8 ×0.90	28.7 N+O 47.6 ×0.60	2.0e0
syn-n4-r5	4 / 5	32.0 N 79.8 ×0.40	10.9 N 19.8 ×0.55	11.0 O 12.0 ×0.92	54.2 N+O 113 ×0.48	1.4e−13
syn-n5-r4	5 / 4	11.4 N 22.2 ×0.51	8.67 N 18.6 ×0.47	36.3 O 65.1 ×0.56	57.0 N+O 108 ×0.53	1.4e−13
syn-n5-r5	5 / 5	36.1 N 81.2 ×0.44	12.9 N 22.7 ×0.57	37.4 O 71.1 ×0.53	87.1 N+O 179 ×0.49	6.0e−12
syn-n5-r6	5 / 6	110 N 259 ×0.43	28.2 N 50.9 ×0.55	47.2 O 85.4 ×0.55	188 N+O 407 ×0.46	2.6e−9
syn-n6-r5	6 / 5	40.6 N 74.3 ×0.55	18.3 N 37.5 ×0.49	310 O 484 ×0.64	371 N+O 607 ×0.61	3.2e−8
syn-n6-r6	6 / 6	113 N 238 ×0.47	26.7 N 46.3 ×0.58	269 O 420 ×0.64	411 N+O 725 ×0.57	9.5e−10
syn-n6-r7	6 / 7	308 N 690 ×0.45	53.5 N 96.8 ×0.55	323 O 535 ×0.60	699 N+O 1353 ×0.52	1.7e−7

2 How the numbers are measured

Test cases. Seven physical one-loop diagrams with a top-quark loop, generated independently by FeynKit (NinjaRS) and MadGraph5_aMC@NLO 3.7.0 (MadLoop), and matched graph by graph: the propagator invariants and masses agree exactly and the integrated results agree up to a constant normalization over all phase-space points. Each diagram is evaluated at three phase-space points (exact rational/radical momenta rounded once to binary64), one fixed helicity configuration, zero widths and $\mu^2 = 100^2$ GeV². Twenty synthetic topologies ($n = 2, \ldots, 6$ propagators, ranks $n-1$, n and $n+1$, random real kinematics with time-like and space-like invariants, real and complex masses, dense random tensor numerators) complete the suite.

NinjaRS. `ninjars pipeline-bench` builds the physical numerator from the DOT graph and the UFO-style model (Feynman rules, wavefunctions, colour projection, Dirac and Lorentz algebra, exact polynomial in the loop momentum), plans and compiles its coefficient heads with SymJIT O2 and then times, per batch of phase-space points: the numerator preparation (wavefunctions, couplings, coefficient heads, Ninja tensor layout and propagator data); the pure reduction (`ninjars::fast` with the numerator outputs and master values of the same evaluation replayed at O(1) cost); the tensor-numerator expansions (live tensor numerator minus pure reduction); the master integrals (the recorded calls replayed through oneloopmaster's native code); and the uninstrumented total. The components add up to the total within 2 %. `ninjars case-bench` performs the same breakdown on the shared bench-case files, which contain exactly the propagators and tensor coefficients given to original Ninja.

MadLoop pipeline. Optimized standalone MadLoop output (`loop_optimized_output`), restricted to the same single loop diagram, Ninja as the only reduction library, no stability rotations, linked against original Ninja 1.2.1 (`g++ -O3 -fcx-fortran-rules`, integral cache disabled) and OneLOop 3.6 (`gfortran -O3`). The numerator time is MadLoop's loop-coefficient construction plus Ninja's tensor-numerator expansions; reduction and master time come from an instrumented Ninja build and are cross-checked by replaying MadLoop's exact Ninja inputs in a C++ driver. The table uses the faster MadLoop variant for every quantity: `GLOBAL_FLAG=-O3` for evaluation and the default flags for compilation time.

Identical-input comparison. The C++ driver feeds original Ninja the same bench cases (`Amplitude<ComplexMasses>`, `TensorNumerator`, explicit S-matrix) and replays recorded numerator outputs and OneLOop results to isolate each component, exactly like the NinjaRS harness. Every replay reproduces the direct evaluation bit for bit.

Environment. One pinned core of a shared AMD EPYC 9754 host, Rust 1.98 (`--profile perf`: opt-level 3, one codegen unit for the hot crates), GCC and gfortran 15.3, no `-march`/`-ffast-math` on either side. Timings are medians of at least seven blocks of at least 20 ms; repeated runs agree within 1–2 %.

3 What is new in NinjaRS

NinjaRS reimplements the integrand-reduction method of Ninja [1], [2] in Rust and embeds it in an end-to-end pipeline built on shared physics and algebra libraries. The original stack used for comparison — MadLoop's numerator [3], Ninja's reduction and OneLOop's master integrals [4] — is a set of independently written Fortran and C++ codes connected through a tensor-numerator interface [5]. NinjaRS replaces each component while keeping Ninja's semantics and results.

3.1 One exact source, from Feynman graph to integrand

Physical numerators are prepared by FeynKit from Feynman-graph DOT files and a UFO-style model: Feynman rules, external wavefunctions, colour projection and Dirac/Lorentz algebra are applied symbolically with Spenso/Idenso and Symbolica. The result is an **exact** polynomial in the loop momentum (and μ^2) whose coefficients are exact expressions in the external kinematics and model parameters. Its distinct coefficient "heads" are deduplicated, optionally factored to remove numerical cancellations, and compiled together into a single multi-output SymJIT evaluator at optimization level O2. Per phase-space point NinjaRS evaluates these heads once and scatters them into Ninja's tensor layout with a precomputed, allocation-free map; MadLoop instead rebuilds its loop polynomial through an open-loop recursion over loop wavefunctions at every point.

Because the polynomial is exact, NinjaRS knows the **true** rank of each numerator. The odd-γ trace removes the q^5 terms of the top-quark pentagons of $gg \to HHH$ and the q^3 terms of the triangles, which NinjaRS therefore reduces at rank 4 and 2, while MadLoop declares ranks 5 and 3 and drives Ninja through more expensive expansions (for the pentagon through its higher-rank $r = n + 1$ path).

3.2 Batching throughout the pipeline

Every NinjaRS interface — numerator coefficients, cut-numerator callbacks, reduction and master integrals — processes a batch of phase-space points together. Callbacks receive one compact batch per cut instead of one call per point and cut; master integrals are requested once per integral family for all cuts and points of a batch. Failures are isolated per phase-space lane: a lane failing a stability check or a master evaluation keeps its previous result while its neighbours are accumulated.

3.3 Generic numerical domains and derivatives

The generic reduction engine is written once for any Symbolica numerical domain: binary64, double-double (`DoubleFloat`), arbitrary-precision `Float` with tracked significant precision, and Symbolica's Dual numbers for first, higher and mixed derivatives with respect to kinematic or model parameters. Precision rescue and derivatives therefore need neither a separate quadruple-precision build nor numerical differencing. Its arithmetic kernels are generated mechanically from Symbolica instruction graphs.

3.4 A dedicated binary64 engine

For ordinary double-precision evaluation, `ninjars::fast` is a monomorphic specialization of the same Laurent reduction, batched over phase-space points, with plain complex arithmetic, precomputed topology data and allocation-free workspaces. It covers every cut level, any number of propagators and ranks up to $n + 1$, with Ninja's default options, skip rules and stability checks, and reproduces original Ninja's results including the split into cut-constructible and rational parts. It is the engine measured in Table 1 and Table 2.

3.5 Master integrals from exact expressions: oneloopmaster

oneloopmaster encodes OneLOop's scalar integrals as exact Symbolica expressions, including analytic-continuation repairs, and generates direct Rust implementations from them. One source therefore serves binary64, double-double and arbitrary precision (and Dual numbers through the interpreter), with lazy conditional branches preserved. The bubble and tadpole tensor coefficients are native ports of OneLOop's routines.

4 Algorithmic improvements

The speed-ups in Table 2 come from changes in how the work is organized, not from different mathematics; NinjaRS and Ninja agree to 10^{-12} or better on well-conditioned points.

- **Topology precomputation.** Cut partitions, complements, parent lists for the coefficient-level subtractions and tadpole reference candidates are built once per topology. Ninja allocates its cut containers and rescans sub-partition relations at every phase-space point.

- **Batched callbacks and masters.** One numerator call per cut (both orientations of a triangle share it) and one master call per family per batch, instead of per point and cut; all master requests of all cuts and points are collected before a single contraction pass.
- **Truncated-jet tensor expansions.** Ninja's `TensorNumerator` materializes symmetric tensor powers of the cut vectors (e_3^r, $e_3^{r-1}a$, ...) and contracts them with the numerator. NinjaRS instead evaluates the homogenized numerator $t^{-R}N(q)$ as one transposed Horner sweep in $u = \frac{1}{t}$ truncated at the requested order, with exact polynomial dependence on μ^2 and x: every tensor coefficient is touched once per expansion. This makes the tensor numerator 2–2.6× faster than Ninja's.
- **Rank-specialized subtractions.** Ninja's FORM-generated coefficient-level subtractions are translated mechanically, and the parent coefficients that vanish identically for the given rank are folded away at compile time (Ninja evaluates every term and discards some afterwards).
- **Shared per-propagator data.** Products of cut-basis vectors with the propagator shifts and their reciprocals are computed once per cut and lane and reused by the Laurent divisions of both orientations and by the subtractions; each uncut denominator costs one reciprocal.
- **Plain complex arithmetic.** Rust's complex multiply and divide have no NaN/infinity recovery, which is what `-fcx-fortran-rules` gives C++; we measure original Ninja with that flag so the comparison is not inflated by `__muldc3`/`__divdc3` calls (worth 20 % on Ninja's side).
- **True numerator rank.** As described above, the exact numerator avoids the inflated ranks MadLoop declares, which roughly triples the cost of the tensor expansions for the pentagon in the MadLoop pipeline.
- **Single-dispatch master kernels.** The generated oneloopmaster code evaluated each Laurent order of C_0 and D_0 separately, repeating the channel and sector selection three times. Hand-transcribed binary64 kernels for the dominant sectors (three-mass triangles, contour boxes and the four-mass Denner–Nierste–Scharf box) perform one dispatch per call, never evaluate inactive branches, and keep the exact expressions as their validated reference; they are 2–3.5× faster than the generated code and about twice as fast as OneLOop.
- **Fast binary64 dilogarithm and native tensor bubbles.** oneloopmaster's binary64 dilogarithm now uses a dedicated series kernel (12–79 ns per call instead of an MPFR detour; contributed to Symbolica as PR #89), and the bubble tensor coefficients $B_0, B_1, B_{11}, B_{111}$ come from one native evaluation sharing the roots and logarithms, 3–5× faster than OneLOop's `olo_b11`.
- **On-shell thresholds in one place.** Exact-zero dispatch in oneloopmaster is combined with OneLOop's on-shell threshold as Ninja configures it, applied once to every master request, and MadLoop's on-shell treatment of the kinematic matrix is reproduced when propagator data are prepared.

Further possible improvements. The reduction is now latency-bound per lane; vectorizing the per-lane arithmetic across phase-space points (structure-of-arrays) is the natural next step. Master requests could be deduplicated across diagrams of a process, and the box and triangle masters could select the cheapest equivalent argument ordering, as OneLOop does internally. For the generic engine, staged fusion of the reduction stages up to master weights reduces the instruction count of an $n = 3$, $r = 3$ reduction from 1,884 to 1,071 operations in an exact inventory.

References

[1] Mastrolia, Pierpaolo, Mirabella, Edoardo, Peraro, and Tiziano, "Integrand reduction of one-loop scattering amplitudes through Laurent series expansion," *JHEP*, vol. 1206, 2012.

[2] T. Peraro, "Ninja: Automated integrand reduction via Laurent expansion for one-loop amplitudes," *Comput. Phys. Commun.*, vol. 185, 2014.

[3] Hirschi *et al.*, "Automation of one-loop QCD corrections," *JHEP*, vol. 1105, 2011.

[4] A. van Hameren, "OneLOop: for the evaluation of one-loop scalar functions," *Comput. Phys. Commun.*, vol. 182, 2011.

[5] Hirschi, Valentin, Peraro, and Tiziano, "Integrand reduction of one-loop amplitudes with a tensor numerator," *JHEP*, vol. 1606, 2016.